

1.

Υλοποιήσαμε

1. Δομές δεδομένων B+ δέντρου

- BPlusMeta (στο block 0): αποθηκεύουμε το TableSchema και το root_block.
- BPlusDataNode (φύλλα):
 - is_leaf
 - num_records
 - next_leaf
 - πίνακας records[] με σταθερό μέγιστο πλήθος (BPLUS_MAX_DATA_RECORDS).
- BPlusIndexNode (εσωτερικοί κόμβοι):
 - is_leaf
 - num_keys
 - πίνακας keys[]
 - πίνακας children[] με num_keys + 1 δείκτες σε blocks.

2. Δημιουργία / άνοιγμα / κλείσιμο αρχείου B+ δέντρου (bplus_file_funcs.c)

- int bplus_create_file(const TableSchema *schema, const char *fileName);
 - Δημιουργεί BF αρχείο.
 - Στο **block 0** γράφει τα BPlusMeta (schema + root_block).
 - Στο **block 1** δημιουργεί ένα *κενό φύλλο* (που είναι αρχικά και ρίζα).
- int bplus_open_file(const char *fileName, int *file_desc, BPlusMeta **metadata);
 - Ανοίγει το αρχείο μέσω BF.
 - Διαβάζει το block 0.
 - Κάνει malloc μια δομή BPlusMeta στη μνήμη και την επιστρέφει στον χρήστη.
- int bplus_close_file(const int file_desc, BPlusMeta* metadata);
 - Ξαναγράφει τα metadata (block 0) από τη μνήμη στο δίσκο.

- Απελευθερώνει τη δομή metadata.
- Κλείνει το αρχείο BF.

3. Εισαγωγή εγγραφής στο B+ δέντρο με split και ενημέρωση ρίζας

- `int bplus_record_insert(const int file_desc, BPlusMeta *metadata, const Record *record);`
- Χρησιμοποιεί τη βοηθητική αναδρομική:
- `static int bplus_insert_recursive(`
- `int file_desc,`
- `const BPlusMeta *meta,`
- `int block_id,`
- `const Record *record,`
- `int *promoted_key,`
- `int *new_block_id,`
- `int *leaf_block_id);`
- Υλοποιεί **όλες τις περιπτώσεις**:
 - Κατέβασμα από τη ρίζα στα φύλλα, μέσω εσωτερικών κόμβων.
 - Εισαγωγή σε φύλλο **χωρίς split** (αν έχει χώρο).
 - **Split φύλλου** όταν είναι γεμάτο:
 - Δημιουργία προσωρινού πίνακα temp με όλες τις παλιές εγγραφές + τη νέα.
 - Ταξινομημένη εισαγωγή στην κατάλληλη θέση.
 - Split στη μέση → αριστερό (παλιό block) + δεξί (νέο block).
 - Ενημέρωση του next_leaf.
 - Προώθηση του *πρώτου κλειδιού* του δεξιού φύλλου προς τα πάνω (promoted_key).
 - Ενημέρωση εσωτερικού κόμβου όταν *παιδί* του κάνει split:
 - Αν χωράει το νέο key ζευγάρι → απλή παρεμβολή.
 - Αν είναι γεμάτος → split εσωτερικού κόμβου:

- Δημιουργία προσωρινών πινάκων temp_keys, temp_children.
 - Εισαγωγή του νέου κλειδιού στη σωστή θέση.
 - Split στη μέση: το μεσαίο κλειδί **ανεβαίνει** στον γονέα.
 - Αριστερός κόμβος = παλιός, δεξιός κόμβος = νέο block.
- Αν γίνει split **στη ρίζα**, η bplus_record_insert:
 - Δεσμεύει **νέο index block**.
 - Το κάνει νέα ρίζα με:
 - 1 key = promoted_key
 - 2 παιδιά: παλιά ρίζα + νέο block από το split.
 - Ενημερώνει metadata->root_block.

4. Αναζήτηση εγγραφής με βάση το key

- int bplus_record_find(const int file_desc, const BPlusMeta *metadata, const int key, Record** out_record);
- Από τη ρίζα:
 - Αν ο κόμβος είναι **εσωτερικός**:
 - Συγκρίνει το key με τα keys[] για να βρει το σωστό child [i].
 - Κατεβαίνει στο αντίστοιχο παιδί.
 - Αν είναι **φύλλο**:
 - Γραμμική αναζήτηση σε leaf->records[] μέχρι να βρει ίδιο key.
 - Αν βρεθεί:
 - Κάνει malloc νέα Record (αν *out_record == NULL).
 - Αντιγράφει την εγγραφή και επιστρέφει 0.
 - Αν δεν βρεθεί: επιστρέφει -1.

5. Χρήση της υπάρχουσας βιβλιοθήκης record και record_generator

- Δεν αλλάξαμε καθόλου:
 - record.h / record.c

- record_generator.h / record_generator.c
- Χρησιμοποιούμε:
 - TableSchema όπως δίνεται.
 - record_get_key(const TableSchema *schema, const Record *rec); για το κλειδί.
 - Οι δομές Record/Schema είναι ακριβώς όπως στην εκφώνηση.

6. Βοηθητικά modules για κόμβους

- Στα bplus_datanode.h/.c και bplus_index_node.h/.c:
 - Ορίσαμε τις δομές BPlusDataNode και BPlusIndexNode.
 - Έχουμε βασικές βοηθητικές συναρτήσεις (για αρχικοποίηση / debugging).
 - Η ουσιαστική προσπέλαση/τροποποίηση των κόμβων γίνεται μέσα από το bplus_file_funcs.c, αλλά οι τύποι και τα constants είναι συγκεντρωμένα στα αντίστοιχα headers.

Δεν υλοποιήσαμε

- Ενημέρωση (update) υπάρχουσας εγγραφής.
- Σύνθετα κλειδιά (χρησιμοποιούμε μόνο το key που δίνει το record_get_key.
- binary search – χρησιμοποιούμε γραμμική, που όμως είναι επαρκής για τα μεγέθη της άσκησης.

2. Παραδοχές που κάναμε

1. Τα κλειδιά (keys) είναι μοναδικά.
 - Αν κατά την εισαγωγή βρούμε στο φύλλο μια εγγραφή με ίδιο key, απορρίπτουμε την εισαγωγή και επιστρέφουμε -1.
2. Το κλειδί κάθε εγγραφής το δίνει η βιβλιοθήκη record.
 - Δεν ξέρουμε εμείς ποιο πεδίο είναι το κλειδί.
 - Καλούμε πάντα:
 - int key = record_get_key(&meta->schema, rec);
 -

3. Δομές κόμβων εγγράφονται αυτούσιες στο block.

- Κάθε block περιέχει είτε έναν BPlusDataNode είτε έναν BPlusIndexNode.
- Το πρώτο πεδίο (is_leaf) είναι κοινό και για τους δύο τύπους, ώστε:
- `int is_leaf = *((int *) data);`

να μας λέει τι κόμβος είναι.

4. μεγέθη κόμβων

- Τα BPLUS_MAX_DATA_RECORDS και BPLUS_MAX_INDEX_KEYS έχουν υπολογιστεί ώστε ο κόμβος να χωράει ακριβώς σε ένα BF_BLOCK_SIZE.
- Δεν αλλάζουμε το BF_BLOCK_SIZE ούτε τη δομή των κόμβων που δίνει η εκφώνηση.

5. Η ρίζα ξεκινά πάντα ως φύλλο στο block 1.

- Στο bplus_create_file:
 - Το block 0 είναι meta.
 - Το block 1 είναι κενό φύλλο.
- Μόνο όταν χρειαστεί split στη ρίζα, δημιουργούμε **νέα** ρίζα (εσωτερικό κόμβο).

6. Δεν αλλάζουμε τη βιβλιοθήκη BF.

- Χρησιμοποιούμε αποκλειστικά τις συναρτήσεις:
 - BF_CreateFile, BF_OpenFile, BF_CloseFile
 - BF_AllocateBlock, BF_GetBlock
 - BF_Block_GetData, BF_Block_SetDirty
 - BF_UnpinBlock, BF_GetBlockCounter

3. Συνοπτική περιγραφή ανά συνάρτηση

static int bplus_get_key(const TableSchema *schema, const Record *rec);

- καλεί record_get_key(schema, rec).
 - Στόχος: να παίρνει το κλειδί μέσω schema.
-

static int bplus_insert_recursive(...)

- Παίρνει:
 - file descriptor BF
 - meta (schema + root_block)
 - τρέχον block_id
 - τη record προς εισαγωγή
 - δύο εξόδους προς τα πάνω: promoted_key, new_block_id
 - leaf_block_id (πού κατέληξε η εγγραφή).

Βήματα:

1. **Διαβάζει τον κόμβο** από το BF:
 - BF_GetBlock(file_desc, block_id, block)
 - data = BF_Block_GetData(block)
 - Ελέγχουμε is_leaf.
2. **Αν είναι φύλλο (BPlusDataNode):**
 - Ελέγχουμε για **διπλό key**: αν βρεθεί, unpin & destroy block, επιστρέφουμε -1.
 - Αν υπάρχει χώρος:
 - Βρίσκουμε ταξινομημένη θέση μέσω μετακίνησης προς τα δεξιά.
 - Εισάγουμε τη record.
 - num_records++, BF_Block_SetDirty, Unpin, Destroy.
 - Γεμίζουμε *leaf_block_id και επιστρέφουμε 0 (no split).
 - Αν **δεν υπάρχει χώρος**:
 - Φτιάχνουμε προσωρινό πίνακα temp με **όλες** τις παλιές εγγραφές + τη νέα στη σωστή θέση.
 - Υπολογίζουμε left_count = total/2, right_count = total - left_count.
 - Γράφουμε τα πρώτα left_count στο παλιό φύλλο.
 - Δεσμεύουμε νέο block για το δεξί φύλλο και γράφουμε τα right_count υπόλοιπα.

- Ενημερώνουμε next_leaf (linked list φύλλων).
- Το up_key = κλειδί της **πρώτης εγγραφής** του νέου φύλλου.
- Unpin & Destroy και τα δύο blocks.
- Επιστρέφουμε 1 (split), γεμίζοντας:
 - *promoted_key = up_key
 - *new_block_id = new_leaf_id
 - *leaf_block_id ανάλογα με το αν το key έπεσε στο αριστερό ή το δεξί φύλλο.

3. Αν είναι εσωτερικός κόμβος (BPlusIndexNode):

- Με βάση τα node->keys[], βρίσκουμε σε ποιο child [i] πρέπει να κατέβουμε.
- Κάνουμε Unpin & Destroy τον τρέχοντα κόμβο.
- Καλούμε αναδρομικά bplus_insert_recursive στο παιδί.
- Αν το παιδί **δεν έκανε split** (res <= 0), απλά επιστρέφουμε την τιμή του.
- Αν το παιδί **έκανε split**:
 - Ξαναδιαβάζουμε τον τρέχοντα κόμβο.
 - Αν χωράει το νέο key/child:
 - Παρεμβολή σε keys[] & children[].
 - num_keys++, SetDirty, Unpin, Destroy, επιστροφή 0.
 - Αν δεν χωράει:
 - Αντιγράφουμε όλα τα keys & children σε προσωρινούς πίνακες.
 - Εισάγουμε το νέο child_promoted_key και child_new_block_id στη σωστή θέση.
 - Υπολογίζουμε mid = total_keys/2.
 - up_key = temp_keys[mid] → αυτό ανεβαίνει στον γονέα.
 - Αριστερός κόμβος = παλιός (keys 0..mid-1).
 - Δεξιός κόμβος = νέο block (keys mid+1..τέλος).
 - Γεμίζουμε *promoted_key, *new_block_id = new_index_id, επιστρέφουμε 1.

int bplus_create_file(const TableSchema *schema, const char *fileName);

- Δημιουργεί το φυσικό BF αρχείο.
- Block 0:
 - Γράφει BPlusMeta μηδενισμένο + schema + root_block = 1.
- Block 1:
 - Ορίζει έναν κενό BPlusDataNode:
 - is_leaf = 1, num_records = 0, next_leaf = -1.

int bplus_open_file(const char *fileName, int *file_desc, BPlusMeta **metadata);

- Ανοίγει το BF αρχείο.
- Διαβάζει το block 0.
- Κάνει malloc(sizeof(BPlusMeta)) και αντιγράφει τα δεδομένα εκεί.
- Επιστρέφει file_desc και pointer σε metadata.

int bplus_close_file(const int file_desc, BPlusMeta* metadata);

- Διαβάζει block 0.
- Αντιγράφει πάνω του τα τωρινά metadata.
- BF_Block_SetDirty, BF_UnpinBlock.
- free(metadata).
- BF_CloseFile(file_desc).

int bplus_record_insert(const int file_desc, BPlusMeta *metadata, const Record *record);

- Καλεί bplus_insert_recursive ξεκινώντας από metadata->root_block.
- Αν res == -1 → σφάλμα / διπλό key.
- Αν res == 0 → split μόνο κάτω από τη ρίζα, όχι στη ρίζα → απλά επιστρέφει leaf_block_id.
- Αν res == 1 → split στη ρίζα:
 - Δεσμεύει νέο block για νέα ρίζα.

- Γράφει 1 key + 2 children.
 - Ενημερώνει metadata->root_block = new_root_id.
 - Επιστρέφει leaf_block_id.
-

int bplus_record_find(...);

- Από metadata->root_block:
 - Όσο βλέπει εσωτερικό κόμβο, κατεβαίνει στο κατάλληλο child.
 - Όταν φτάσει σε φύλλο:
 - Γραμμικά ψάχνει σε leaf->records.
 - Αν βρει → επιστρέφει 0 και γυρίζει *out_record.
 - Αν όχι → -1.